

TP3 : Browser Fingerprinting

Web Privacy

Houda Toudali

UM6P - College of Computing

1. Objective

The goal of this TP was to see how a browser can still be recognized without depending only on cookies. I built a small local Flask application and collected information in several ways: first from the HTTP request, then with JavaScript, and finally from a simple typing experiment.

The project was tested locally on 127.0.0.1:8000. I also compared normal browsing with Incognito mode, third-party cookie blocking and Privacy Badger.

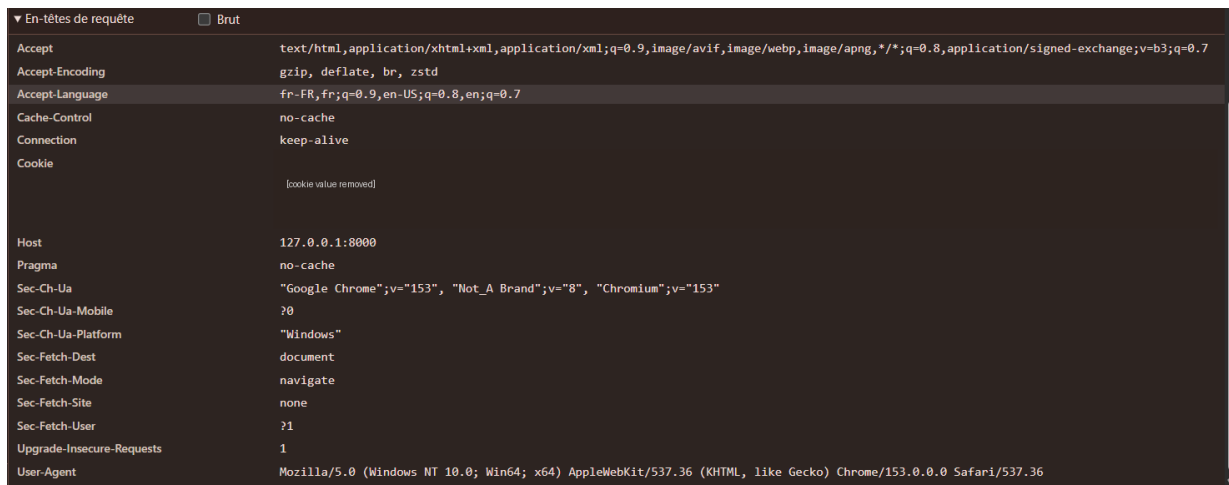
2. Passive fingerprinting

I started with information that is already sent by the browser in the HTTP request. No JavaScript was needed for this part. The Flask server printed the client address, HTTP method, User-Agent and preferred language.

```
===== NEW VISIT =====
IP address : 127.0.0.1
HTTP method : GET
User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
Accept-Language : fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
=====
```

Figure 1: First passive values in Flask.

The first request showed Chrome on Windows and the language preference **fr-FR**. I then inspected the request directly in DevTools and added more headers to the collector, such as **Accept**, **Accept-Encoding**, **Sec-CH-UA**, **Sec-CH-UA-Platform**, **Sec-Fetch-Site** and **Upgrade-Insecure-Requests**.



▼ En-têtes de requête	☐ Brut
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Cache-Control	no-cache
Connection	keep-alive
Cookie	[cookie value removed]
Host	127.0.0.1:8000
Pragma	no-cache
Sec-CH-UA	"Google Chrome";v="153", "Not_A_Brand";v="8", "Chromium";v="153"
Sec-CH-UA-Mobile	?0
Sec-CH-UA-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

Figure 2: Request headers in Chrome.

These headers reveal several details about the browser, supported formats, platform and request context. The cookie value is not useful for this report, so it is hidden in the figure.

I repeated the same request in an Incognito window. The passive fingerprinting values stayed the same.

```

===== NEW VISIT =====
IP address : 127.0.0.1
HTTP method : GET
User-Agent : Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36
Accept-Language : fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Accept : text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding : gzip, deflate, br, zstd
Sec-CH-UA : "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
Sec-CH-UA-Platform : "Windows"
Sec-Fetch-Site : none
Upgrade-Insecure-Requests : 1
=====

```

Figure 3: Passive values in Incognito mode.

In DevTools, the Incognito request did not contain the previous cookie, but the other browser headers were still present.

▼ En-têtes de requête ☐ Brut	
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	fr-FR,fr;q=0.9,en-US;q=0.8,en;q=0.7
Cache-Control	no-cache
Connection	keep-alive
Host	127.0.0.1:8000
Pragma	no-cache
Sec-Ch-UA	"Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"
Sec-Ch-UA-Mobile	?0
Sec-Ch-UA-Platform	"Windows"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

Figure 4: Incognito request headers.

This was the first clear difference between stateful and stateless tracking in the experiment. The cookie state changed, but the browser characteristics could still be observed.

3. Active feature collection

For the next part I used JavaScript. I first tested a simple value with `navigator.language` and displayed it on the page.

Browser Fingerprinting Lab

Passive Feature Collection

Passive features are collected from the HTTP request.

Active Feature Collection

Browser language: fr-FR

Figure 5: First active feature.

Then I extended the script to collect the language, number of logical CPU cores, screen resolution, color depth, browser window size, time zone and referrer.

Browser Fingerprinting Lab

Passive Feature Collection

Passive features are collected from the HTTP request.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 488x730
Time zone: Africa/Casablanca
Referrer: None

Figure 6: Active features shown on the page.

On my browser the main values were fr-FR, 20 logical cores, a screen resolution of 1536x864, color depth 24 and the time zone Africa/Casablanca. The window size changed when I resized DevTools, so I considered it less stable than the other values.

The JavaScript values were sent as JSON to the Flask server.

```
===== ACTIVE FEATURES =====  
Language : fr-FR  
CPU cores : 20  
Screen resolution : 1536x864  
Color depth : 24  
Window size : 488x730  
Time zone : Africa/Casablanca  
Referrer : None  
=====
```

Figure 7: Active values received by Flask.

I also checked the POST request in the Network tab to confirm that the same data was really sent by the browser.

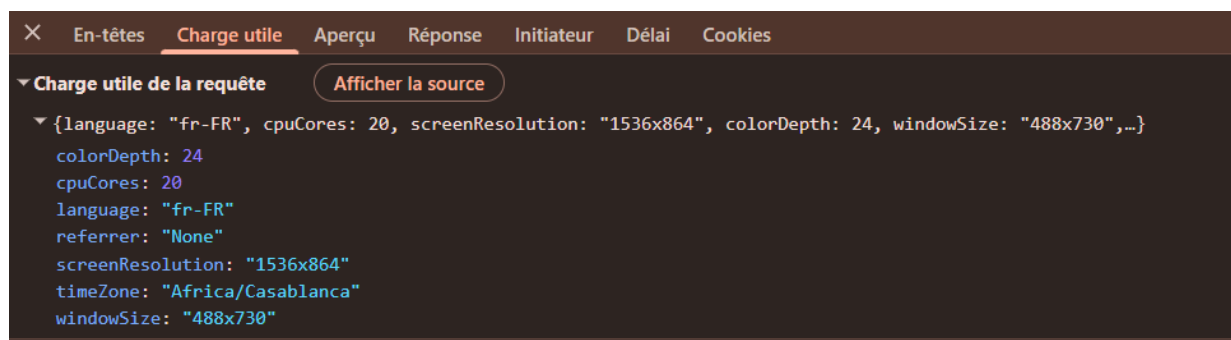


Figure 8: POST payload with the active values.

Privacy tests

I repeated the active collection in Incognito mode. The language, CPU cores, screen resolution, color depth and time zone stayed the same. Only the browser window size changed because the window and DevTools layout were different.



Browser Fingerprinting Lab

Passive Feature Collection

Passive features are collected from the HTTP request.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 154x730
Time zone: Africa/Casablanca
Referrer: None

Figure 9: Active features in Incognito mode.

I then blocked third-party cookies. The active features were still accessible because this JavaScript collection does not depend on a cookie.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 488x730
Time zone: Africa/Casablanca
Referrer: None

Figure 10: Active features with cookie blocking.

Finally I enabled Privacy Badger and repeated the test. The same first-party script still collected the same values on the local page.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 488x730
Time zone: Africa/Casablanca
Referrer: None

Figure 11: Active features with Privacy Badger.

In this local experiment, these protections helped mainly with stored state or third-party tracking. They did not stop the first-party script from reading the browser properties used here.

4. Typing behaviour

The next part was a small behavioural fingerprint. The user had to type the sentence:

Cookies are gone, but Alice's rhythm still leaves traces!

The timer started on the first key press. Backspace presses were counted as corrections, and the test stopped only when the typed text matched the target sentence. Typing speed was calculated as the number of characters divided by the total time in seconds.

For the final manual run, I obtained 29.68 seconds, 1.92 characters per second and one correction.

Browser Fingerprinting Lab

Passive Feature Collection

Passive features are collected from the HTTP request.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 719x730
Time zone: Africa/Casablanca
Referrer: None

Browser Fingerprint

b6a061e4110ebcec104a7fd5687324c8411b44a42345e2e22536104542c81c0a

Alice's Typing Rhythm

Type the following sentence exactly:

Cookies are gone, but Alice's rhythm still leaves traces!

Total time: 29.68 seconds
Typing speed: 1.92 characters/second
Corrections: 1

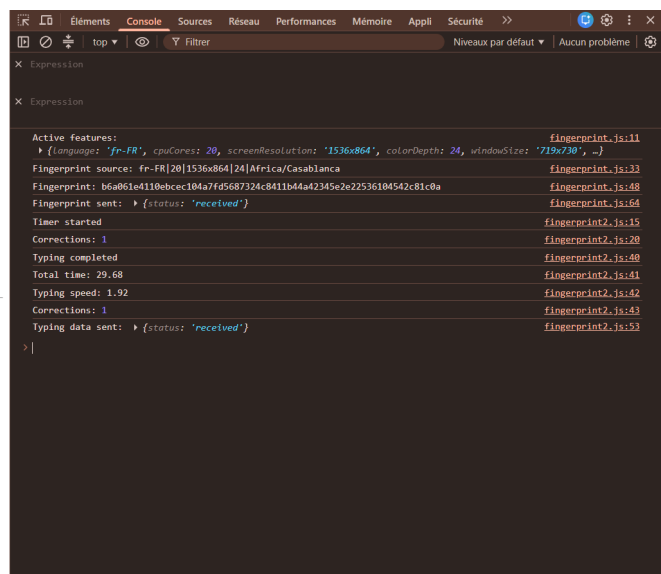


Figure 12: Typing test result.

The same values were received by Flask.

```
===== TYPING BEHAVIOUR =====  
Total time : 29.68 seconds  
Typing speed : 1.92 characters/second  
Corrections : 1  
=====
```

Figure 13: Typing values received by Flask.

The Network tab also showed the three values in the `/collect-typing` request.



Figure 14: Typing POST payload.

This measurement is not a fixed identity. Typing speed can change with fatigue, keyboard, posture or just the way the user types during one attempt.

5. SHA-256 browser fingerprint

For the last part I combined a few values in a fixed order and generated a SHA-256 hash in the browser. I used language, CPU cores, screen resolution, color depth and time zone. I did not include typing behaviour or window size because both can change easily.

The source string was equivalent to:

```
fr-FR|20|1536x864|24|Africa/Casablanca
```

The generated fingerprint was:

```
b6a061e4110ebcec104a7fd5687324c8411b44a42345e2e22536104542c81c0a
```

Browser Fingerprinting Lab

Passive Feature Collection

Passive features are collected from the HTTP request.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 460x730
Time zone: Africa/Casablanca
Referrer: None

Browser Fingerprint

b6a061e4110ebcec104a7fd5687324c8411b44a42345e2e22536104542c81c0a

Alice's Typing Rhythm

Type the following sentence exactly:

Cookies are gone, but Alice's rhythm still leaves traces!

Figure 15: Generated SHA-256 fingerprint.

I reloaded the page several times and obtained the same hash. I also opened the page in Incognito mode and the fingerprint was still the same because the selected input values did not change.

Browser Fingerprinting Lab

Passive Feature Collection

Passive features are collected from the HTTP request.

Active Feature Collection

Language: fr-FR
CPU cores: 20
Screen resolution: 1536x864
Color depth: 24
Window size: 892x730
Time zone: Africa/Casablanca
Referrer: None

Browser Fingerprint

b6a061e4110ebcec104a7fd5687324c8411b44a42345e2e22536104542c81c0a

Alice's Typing Rhythm

Type the following sentence exactly:

Cookies are gone, but Alice's rhythm still leaves traces!

Figure 16: Same fingerprint in Incognito mode.

For the final request, the raw features were no longer sent to the server. The browser computed the hash locally and Flask received only the fingerprint value.

```
===== BROWSER FINGERPRINT =====  
Fingerprint : b6a061e4110ebcec104a7fd5687324c8411b44a42345e2e22536104542c81c0a  
=====
```

Figure 17: Fingerprint received by Flask.

I verified the last request in DevTools. Its payload contained only the field `fingerprint`.

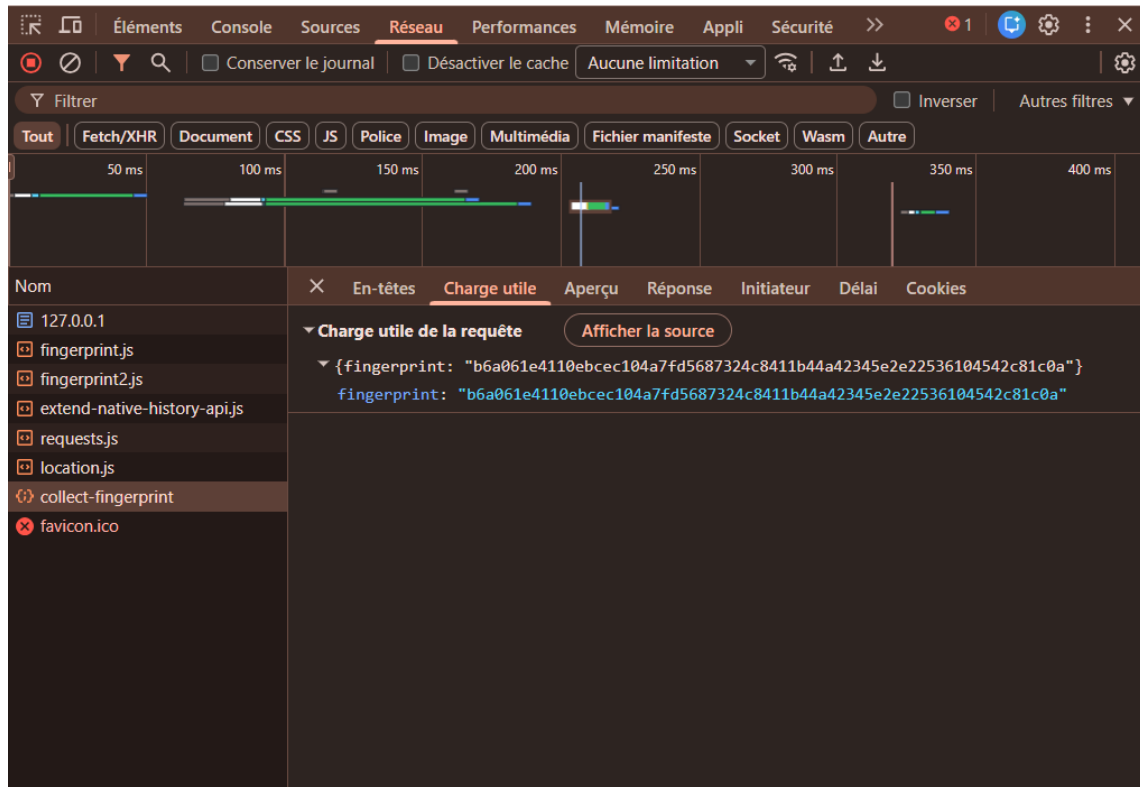


Figure 18: Final request containing only the hash.

6. Conclusion

This TP showed that deleting or blocking cookies is not enough to remove every way of recognizing a browser. HTTP headers already expose passive clues, and JavaScript can collect more information from the browser and device.

The tests with Incognito mode, cookie blocking and Privacy Badger did not change the main active values used by my local fingerprint collector. The typing experiment also showed that behaviour can be another signal, but it is less stable. Finally, the stable features were enough to generate the same SHA-256 fingerprint across normal reloads and Incognito mode.